

Objetivo

Identificar labels y selectors utilizados por Deployments, ReplicaSets y Pods. Analizar ReplicaSets activos e históricos generados por rolling updates y rollback. Diagnosticar un estado inesperado provocado por un Pod que dejó de coincidir con el selector del controlador. Corregir únicamente el recurso anómalo y restaurar el estado esperado del workload.

Requisitos previos

- Haber completado la Práctica 2 y conservar el Deployment webapp y sus ReplicaSets.
- Tener Docker Desktop y Minikube en ejecución.
- Tener kubectl configurado contra el clúster correcto.
- Utilizar Visual Studio Code con Git Bash como terminal integrada.
- Disponer del archivo lab2-1-scenario.sh proporcionado con esta práctica.

Duración estimada

20 minutos

Introducción

En esta práctica complementaria recibirás el Deployment webapp en un estado inesperado después de aplicar un cambio externo. La aplicación continúa funcionando, pero el número y clasificación de Pods ya no coincide con lo esperado. Deberás utilizar labels, selectors, ReplicaSets y ownerReferences para reconstruir qué ocurrió, identificar el recurso fuera del conjunto administrado y restaurar el estado correcto sin modificar el Deployment.

Instrucciones



Escenario

El Deployment `webapp` funcionaba correctamente al finalizar la Práctica 2.

Después de un cambio realizado por otra persona, el workload presenta un comportamiento inesperado:

- el Deployment continúa disponible;

- los Pods principales continúan **Running** ;
- existe evidencia de que Kubernetes creó una réplica para conservar el estado deseado;
- hay un Pod que ya no aparece al filtrar por el selector habitual de la aplicación.

Tu objetivo es descubrir:

¿Qué cambió?
¿Por qué Kubernetes reaccionó creando otra réplica?
¿Qué recurso quedó fuera del conjunto administrado?
¿Cuál es la corrección mínima para volver al estado estable?

 Copiar

Distribución sugerida

Preparación	2 min
Reto 1	6 min
Reto 2	7 min
Reto 3	5 min
Total	20 min

 Copiar

Preparación del escenario

Paso 1. Confirmar el estado base

- **Paso 1.** Ubícate en el directorio utilizado por la Práctica 2 y confirma que **webapp** continúa disponible.

```
cd /c/LABS/kubernetes/lab2
```

 Copiar

```
kubectl get deployment webapp  
kubectl get replicaset -l app=webapp  
kubectl get pods -l app=webapp
```

Resultado esperado:

El Deployment debe tener sus réplicas disponibles y deben existir ReplicaSets asociados con la aplicación.

IMPORTANTE: Si no existen ReplicaSets históricos, revisa que hayas completado previamente el rolling update y rollback del Lab 3.

Paso 2. Aplicar el cambio que originó el incidente

- **Paso 2.** Descarga el archivo en la siguiente ruta.

```
curl -L -o lab2-1-scenario.sh  
https://raw.githubusercontent.com/Netec-  
Mx/CKA/refs/heads/main/labs/lab4/lab2-1-  
scenario.sh
```

[Copiar](#)

- **Paso 3.** Ejecuta el escenario proporcionado sin abrir el archivo ni revisar previamente su contenido.

```
bash lab2-1-scenario.sh
```

[Copiar](#)

- **Paso 4.** Espera unos segundos para permitir que el controlador reaccione al cambio aplicado.

```
sleep 5
```

[Copiar](#)

IMPORTANTE: No utilices `cat`, VS Code ni otro editor para inspeccionar `lab2-1-scenario.sh` antes de completar el diagnóstico.

Reto 1. Reconstruir el estado del workload

Tiempo sugerido: 6 minutos

No modifiques recursos todavía. Primero determina qué objetos existen y cómo se relacionan.

Reto 1.1. Revisar el Deployment

- **Paso 5.** Comprueba el estado deseado y disponible de `webapp`.

```
kubectl get deployment webapp
```

[Copiar](#)

Responde:

¿Cuántas réplicas desea el Deployment?

[Copiar](#)

¿Cuántas aparecen Ready?

¿El Deployment está realmente degradado?

Reto 1.2. Analizar los Pods visibles con el selector habitual

- **Paso 6.** Consulta únicamente los Pods que todavía coinciden con `app=webapp`.

```
kubectl get pods \  
-l app=webapp \  
-o wide
```

[Copiar](#)

- **Paso 7.** Consulta después todos los Pods del namespace sin selector.

```
kubectl get pods \  
--show-labels
```

[Copiar](#)

Preguntas de análisis

1. ¿Aparecen más Pods en la segunda consulta?
2. ¿Existe algún Pod Running que no aparezca con `-l app=webapp`?
3. ¿Qué label diferencia ese Pod del resto?
4. ¿El nombre del Pod sugiere que pertenecía originalmente a webapp?

[Copiar](#)

Reto 1.3. Identificar propietarios

- **Paso 8.** Consulta `ownerReferences` para diferenciar Pods administrados de recursos que quedaron fuera del conjunto efectivo.

```
kubectl get pods \
-o custom-
columns=NAME:.metadata.name,APP:.metadata.labels
.app,OWNER:.metadata.ownerReferences[0].name
```

[Copiar](#)

Evidencia requerida

Debes poder señalar:

Pods actualmente contados por el ReplicaSet.
Pod que continúa existiendo pero ya no coincide con el selector.
ReplicaSet propietario original del Pod anómalo.

[Copiar](#)

Reto finalizado

Resultado esperado: Estado del Deployment, ReplicaSets y Pods analizado correctamente, identificando labels, selectors y relaciones de administración.



Reto 2. Explicar por qué apareció una réplica adicional

Tiempo sugerido: 7 minutos

Ahora relaciona el comportamiento observado con los selectors y el estado deseado.

Reto 2.1. Identificar selector y Pod template

- **Paso 9.** Obtén el selector estable del Deployment.

```
kubectl get deployment webapp \
  -o jsonpath='{.spec.selector.matchLabels}'
{"\n"}
```

[Copiar](#)

- **Paso 10.** Consulta los labels definidos en el Pod template actual.

```
kubectl get deployment webapp \
  -o jsonpath='{.spec.template.metadata.labels}'
{"\n"}
```

[Copiar](#)

Reto 2.2. Analizar ReplicaSets activos e históricos

- **Paso 11.** Muestra todos los ReplicaSets de `webapp` y diferencia el activo de las revisiones históricas.

```
kubectl get replicaset \
  -l app=webapp \
  --show-labels
```

[Copiar](#)

- **Paso 12.** Consulta el historial del Deployment.

```
kubectl rollout history deployment/webapp
```

[Copiar](#)

Preguntas del reto

1. ¿Cuál ReplicaSet tiene réplicas activas?
2. ¿Cuáles tienen DESIRED=0?
3. ¿Qué pod-template-hash comparten los Pods activos?
4. ¿El Pod anómalo conserva ownerReference hacia un ReplicaSet?
5. ¿Por qué el controlador creó otro Pod si el Pod anómalo seguía Running?

[Copiar](#)

Reto 2.3. Formular la causa raíz

No avances hasta poder completar esta explicación:

El ReplicaSet considera como miembros únicamente los Pods que _____.

[Copiar](#)

Cuando un Pod deja de coincidir con _____, deja de contar para el estado deseado aunque continúe _____.

Por eso el ReplicaSet crea _____ para volver a alcanzar el número de réplicas configurado.

Pistas permitidas

```
kubectl get pods --show-labels
kubectl get replicaset --show-labels
kubectl describe replicaset
kubectl rollout history
kubectl get ... -o jsonpath
```

[Copiar](#)

Reto finalizado

Resultado esperado: Pod fuera del selector detectado mediante labels, `ownerReferences` y `pod-template-hash`, explicando por qué el ReplicaSet creó una réplica de reemplazo.

Reto 3. Restaurar el estado estable

Tiempo sugerido: 5 minutos

Debes corregir el incidente sin cambiar la definición del Deployment.

Restricciones

```
[ ] No elimines webapp.  
[ ] No modifiques spec.selector del Deployment.  
[ ] No reduzcas replicas.  
[ ] No cambies los labels del Pod template.  
[ ] No elimines ReplicaSets históricos.
```

[Copiar](#)

Reto 3.1. Identificar el recurso que debe retirarse

- **Paso 13.** Localiza el Pod que permanece fuera del selector `app=webapp`.

Pistas permitidas

```
kubectl get pods --show-labels  
kubectl get pods -l app=webapp  
kubectl get pods -l 'app!=webapp'
```

[Copiar](#)

NOTA: El filtro negativo puede incluir otros Pods del namespace. Utiliza también nombre, ownerReference y labels para identificar el recurso correcto.

Reto 3.2. Aplicar la corrección mínima

- **Paso 14.** Elimina únicamente el Pod que quedó fuera del conjunto administrado.

No se proporciona el comando exacto: debes construirlo a partir del nombre identificado durante el diagnóstico.

Reto 3.3. Validar recuperación

- **Paso 15.** Confirma que únicamente permanecen los Pods administrados con `app=webapp`.

```
kubectl get pods \  
-l app=webapp \  
--show-labels
```

[Copiar](#)

- **Paso 16.** Revisa el ReplicaSet activo.

```
kubectl get replicaset \  
-l app=webapp
```

[Copiar](#)

- **Paso 17.** Confirma que el Deployment se encuentra estable y sin operaciones pendientes.

```
kubectl rollout status deployment/webapp
```

[Copiar](#)

Validación final

La práctica queda completada cuando puedes demostrar:

```
[✓] webapp conserva el número deseado de réplicas.  
[✓] Todos los Pods administrados tienen app=webapp.  
[✓] No queda ningún Pod aislado del incidente.  
[✓] El ReplicaSet activo tiene DESIRED=CURRENT=READY.  
[✓] Los ReplicaSets históricos permanecen  
conservados.  
[✓] El Deployment no fue recreado ni alterado para  
resolver el problema.
```

[Copiar](#)

Ejecuta:

```
echo "=== Validacion final Lab 2.1 ==="
```

 Copiar

```
echo ""
echo "--- Deployment ---"
kubectl get deployment webapp

echo ""
echo "--- Pods administrados ---"
kubectl get pods \
  -l app=webapp \
  --show-labels

echo ""
echo "--- ReplicaSets ---"
kubectl get replicaset \
  -l app=webapp

echo ""
echo "--- Historial ---"
kubectl rollout history deployment/webapp

echo ""
echo "--- Estado del rollout ---"
kubectl rollout status deployment/webapp

echo ""
echo "=== Fin de validacion ==="
```



Cierre del reto

El comportamiento observado demuestra que un controlador no cuenta Pods simplemente porque:

```
están Running
tienen un nombre parecido
fueron creados originalmente por la aplicación
```

 Copiar

La pertenencia efectiva depende de:

```
labels del recurso
+
selector del controlador
```

 Copiar

Cuando la coincidencia desaparece:

```
Pod sigue Running
↓
deja de coincidir con selector
↓
ReplicaSet ya no lo cuenta
↓
estado actual < estado deseado
↓
ReplicaSet crea reemplazo
```

 Copiar

El diagnóstico correcto consiste en entender esa relación antes de modificar recursos.



Resolución de problemas

Problema 1. El script indica que no encontró Pods

El escenario depende de `webapp`.

Comprueba:

```
kubectl get deployment webapp
kubectl get pods -l app=webapp
```

 Copiar

Si no existen, debes recuperar primero el estado final de la Práctica 2.

Problema 2. Solo observas el número normal de Pods

Consulta todos los Pods sin aplicar selector:

```
kubectl get pods --show-labels
```

 Copiar

Recuerda que un Pod puede seguir **Running** aunque ya no sea contado por el ReplicaSet.

Problema 3. No aparecen ReplicaSets históricos

Comprueba:

```
kubectl rollout history deployment/webapp
kubectl get replicaset -l app=webapp
```

[Copiar](#)

Si el Deployment fue recreado después del Lab 3, las revisiones anteriores ya no estarán disponibles.

Problema 4. Eliminaste accidentalmente un Pod administrado

No crees uno manualmente.

El ReplicaSet debe restaurar automáticamente la cantidad deseada:

```
kubectl rollout status deployment/webapp
kubectl get pods -l app=webapp
```

[Copiar](#)

Espera a que el controlador complete la reconciliación.

¡Felicidades! Has completado el Laboratorio.

Resultado final

Al finalizar el reto habrás diagnosticado una anomalía de pertenencia entre Pods y ReplicaSets, identificado el efecto de labels y selectors sobre el estado deseado y restaurado el workload eliminando únicamente el recurso que quedó fuera del control del Deployment.

Notas y consideraciones

- Esta práctica está diseñada para completarse en 20 minutos y reutiliza los recursos creados en el Lab 2.
- No abras ni inspecciones lab2-1-scenario.sh antes de iniciar el reto.
- No elimines ni recrees el Deployment webapp.
- No cambies el selector del Deployment para resolver el incidente.
- Esta práctica complementaria no incluye prompts de apoyo con IA.

URL de referencia

- [Labels and Selectors](#)
- [ReplicaSet](#)
- [Deployments](#)